Term Project: Instant Messaging App

Test Plan Document

Table of Contents

1 Introduction.....

1.1 Purpose.....

1.2 Target Audience.....

1.3 Terms and Definitions.....

2 Test Plan Description.....

2.1 Scope of Testing.....

2.2 Testing Schedule.....

2.3 Release Criteria.....

3 Unit Testing.....

3.1 Input Validation.....

3.2 Password Hashing.....

3.3 JWT Token Utility.....

3.4 Email Verification Code.....

4 Feature Testing.....

4.1 User Signup.....

4.2 Email Verification.....

4.3 User Login.....

4.4 User Logout.....

4.5 Get User Info.....

4.6 Update Profile.....

4.7 Search Contacts.....

4.8 Get All Contacts.....

4.9 Get Contacts for List.....

4.10 Delete Direct Messages .....
4.11 Get Messages .....
4.12 Delete Message .....
5 System Testing.....

1 Introduction

This document defines the complete test plan for the backend implementation of the instant messaging application. It is written in support of a Test-Driven Development (TDD). The document covers the scope of testing, testing schedule, release criteria, and the full set of test cases across unit, feature, integration, and system levels.

1.1 Purpose

This document serves two purposes. First, it supports a TDD workflow by defining all expected behaviors. Each test case specifies precise inputs and expected outputs so that the backend can be built to satisfy them. Second, it provides a reference for verifying that the completed system meets its functional and security requirements before deployment and integration with the provided frontend.

1.2 Target Audience

This document is intended for the developer, the course instructor and TA who will evaluate the project.

1.3 Terms and Definitions

TDD: Test-Driven Development — tests are written before implementation code.
JWT: JSON Web Token — a signed token used to authenticate and authorize users.
HTTP-only Cookie: A browser cookie inaccessible to JavaScript, used to store the JWT securely.
Jest: A JavaScript testing framework used to write and run all test cases.
Supertest: A Node.js library for making HTTP assertions against an Express server in tests.
mongodb-memory-server: A library that spins up an in-memory MongoDB instance for isolated, fast testing.
Socket.IO: A library enabling real-time, bidirectional WebSocket communication.
TTL: Time-To-Live — the duration before a value (e.g. a verification code) expires.

XSS: Cross-Site Scripting — an injection attack that inserts malicious scripts into web content.

NoSQL Injection: An attack that manipulates MongoDB query operators (e.g. $gt) to bypass logic.

MERN: MongoDB, Express, React, Node.js — the technology stack used for this project.

2 Test Plan Description

This section defines the overall scope of testing, the schedule under which testing will be conducted, and the release criteria that must be satisfied before the backend is considered ready for integration with the frontend.

2.1 Scope of Testing

This test plan covers the complete backend of the instant messaging application. Testing is organized into four layers: Unit Testing (individual utility functions including JWT generation and verification, password hashing, input validation, and email verification code generation), Feature Testing (every REST API endpoint tested end-to-end against an in-memory MongoDB instance using Jest and Supertest, covering both success and failure paths), System Testing (full multi-step user flows and WebSocket real-time messaging behavior using live Socket.IO client connections), and Security Testing (JWT authorization enforcement, NoSQL injection resistance, XSS payload handling, and brute force rate limiting as a stretch goal).

2.2 Testing Schedule

Phase 1 — Setup: Configure Jest, Supertest, and mongodb-memory-server; establish test scaffold.
Phase 2 — Unit Tests: Write and pass all unit tests for JWT, bcrypt, validation, and email code utilities.
Phase 3 — Feature Tests: Write and pass all API integration tests for Auth, Contacts, and Messages routes.
Phase 4 — System Tests: Write and pass WebSocket tests and full end-to-end scenario tests.
Phase 5 — Security: Write and pass all security tests; document any known gaps.
Phase 6 — Final Review: All tests passing; submit test plan document.

2.3 Release Criteria

The backend is considered ready for deployment when all of the following criteria are met: 100% of Unit Tests pass with no failures; 100% of Feature Tests pass with no failures; 100% of System and WebSocket Tests pass with no failures; 100% of Security Tests pass; no test produces an unhandled exception or server crash; all protected routes enforce JWT authentication with zero unauthorized access.

3 Unit Testing

Unit testing verifies the correctness of individual, isolated functions and modules before they are integrated into the larger system. Since unit tests do not involve the database, network, or other external dependencies, they run fast and pinpoint failures precisely. The testing framework used is Jest. All utilities are tested in isolation via pure function calls with no server or database running.

3.1 Input Validation

This unit covers the validation logic applied to incoming signup request data before any database operation is performed. The validator must reject invalid or missing inputs and accept well-formed ones. Minimum criteria for release: all 5 test cases pass.

3.1.1 TC1.1 — Empty email field is rejected

Input: email: “”, password: “password123”. Expected output: validation fails with an email-required error. Pass criteria: the function returns an error or false for an empty email field.

3.1.2 TC1.2 — Empty password field is rejected

Input: email: “user@test.com”, password: “”. Expected output: validation fails with a password-required error. Pass criteria: the function returns an error or false for an empty password field.

3.1.3 TC1.3 — Password under 8 characters is rejected

Input: email: “user@test.com”, password: “abc123” (6 characters). Expected output: validation fails with a password-too-short error. Pass criteria: the function returns an error or false. Minimum password length is 8 characters.

3.1.4 TC1.4 — Invalid email format is rejected

Input: email: “notanemail”, password: “password123”. Expected output: validation fails with an invalid email format error. Pass criteria: the function returns an error or false for a malformed email string.

3.1.5 TC1.5 — Valid email and password pass validation

Input: email: “user@test.com”, password: “securePass1”. Expected output: validation passes with no error returned. Pass criteria: the function returns valid or true.

3.2 Password Hashing

This unit tests the bcrypt hashing and comparison utilities. Passwords must never be stored in plaintext. Hashing must produce an irreversible output, and comparison must correctly distinguish matching from non-matching inputs. Minimum criteria for release: all 3 test cases pass.

3.2.1 TC2.1 — Hashing a password returns a non-plaintext string

Input: the string “password123”. Expected output: a hash string that does not equal the original input. Pass criteria: hash !== original plaintext. The hash must be a valid bcrypt hash string.

3.2.2 TC2.2 — Correct plaintext matches its hash

Input: plaintext “password123” and its corresponding bcrypt hash. Expected output: bcrypt.compare returns true. Pass criteria: the comparison resolves to true.

3.2.3 TC2.3 — Wrong plaintext does not match a hash

Input: plaintext “wrongPassword” and the bcrypt hash of “password123”. Expected output: bcrypt.compare returns false. Pass criteria: the comparison resolves to false.

3.3 JWT Token Utility

This unit tests the JWT generation and verification functions. Tokens must be signed correctly, carry the right payload, and reject expired or tampered inputs. These functions

underpin the security of every protected route. Minimum criteria for release: all 4 test cases pass.

3.3.1 TC3.1 — Generating a token with a valid userId returns a signed JWT string

Input: userId “abc123”. Expected output: a non-empty JWT string. Pass criteria: the token is a string containing exactly three dot-separated segments, confirming it is a well-formed JWT.

3.3.2 TC3.2 — Verifying a valid token returns the correct decoded payload

Input: a freshly generated token for userId “abc123”. Expected output: the decoded payload contains { userId: “abc123” }. Pass criteria: payload.userId === “abc123”.

3.3.3 TC3.3 — Verifying an expired token throws an error

Input: a token generated with an expiry of -1 second (already expired at creation time). Expected output: the verify function throws a TokenExpiredError or equivalent. Pass criteria: the call throws and is caught in the catch block.

3.3.4 TC3.4 — Verifying a tampered token throws an error

Input: a valid token with its signature segment altered by one character. Expected output: the verify function throws a JsonWebTokenError or equivalent. Pass criteria: the call throws and is caught in the catch block.

3.4 Email Verification Code

This unit tests the utility that generates one-time email verification codes. Codes must be numeric, of a fixed length of 6 digits, reasonably random, and must expire after their TTL window. Minimum criteria for release: all 3 test cases pass.

3.4.1 TC4.1 — Generated code is a 6-digit numeric string

Input: a call to generateVerificationCode(). Expected output: a string of exactly 6 numeric digits, for example “483920”. Pass criteria: the code matches the regex.

3.4.2 TC4.2 — Two separately generated codes are not identical

Input: two separate calls to generateVerificationCode(). Expected output: the two returned values are different strings. Pass criteria: code1 !== code2. This is a basic randomness sanity check.

3.4.3 TC4.3 — A code expires after its TTL window

Input: a code generated with a TTL of 1 second, then verified after 2 seconds. Expected output: isCodeValid() returns false after the TTL has elapsed. Pass criteria: the code is rejected as expired. In production the TTL is 15 minutes; a short TTL is used in the test environment.

4 Feature Testing

Feature testing (integration testing) validates that each API endpoint behaves correctly end-to-end, from receiving an HTTP request through business logic and middleware to database interaction and response. Tests are written using Jest and Supertest against a live Express server connected to an in-memory MongoDB instance (mongodb-memory-server). The database is seeded before relevant tests and wiped between suites to ensure isolation. This is an interaction test: we are verifying that each endpoint correctly processes inputs and produces the expected outputs as specified in the API contract.

4.1 User Signup (POST /api/auth/signup)

Signup is the entry point for all new users. It validates inputs, ensures email uniqueness, hashes the password, creates the user record, and triggers a verification email. It must not create an account for invalid data or duplicate emails. Minimum criteria for release: all 7 test cases pass.

4.1.1 TC5.1 — Valid email and password creates a new user

Input: POST body { email: “new@test.com”, password: “securePass1” }. Expected output: 201 Created; user object returned with id, email, and profileSetup: false. Pass criteria: status 201 and body.user.email === “new@test.com”.

4.1.2 TC5.2 — Missing email returns 400

Input: POST body { password: “securePass1” } with no email field. Expected output: 400 Bad Request. Pass criteria: status 400.

4.1.3 TC5.3 — Missing password returns 400

Input: POST body { email: “new@test.com” } with no password field. Expected output: 400 Bad Request. Pass criteria: status 400.

4.1.4 TC5.4 — Password under 8 characters returns 400

Input: POST body { email: “new@test.com”, password: “abc” }. Expected output: 400 Bad Request. Pass criteria: status 400.

4.1.5 TC5.5 — Invalid email format returns 400

Input: POST body { email: “notanemail”, password: “securePass1” }. Expected output: 400 Bad Request. Pass criteria: status 400.

4.1.6 TC5.6 — Duplicate email returns 409

Input: POST body using an email already registered in the seeded database. Expected output: 409 Conflict. Pass criteria: status 409.

4.1.7 TC5.7 — Valid signup triggers the email verification service

Input: POST body { email: “new@test.com”, password: “securePass1” }. Expected output: the email service mock is called exactly once with the correct address. Pass criteria: the mocked emailService is called with “new@test.com”. The email service is mocked so no actual SMTP delivery occurs.

4.2 Email Verification (POST /api/auth/verify-email)

After signup, the user must verify their email by submitting the 6-digit code sent to them. This endpoint activates the account and allows the user to proceed to login. An unverified user must not be able to log in. This endpoint is not in the original API spec and is a new endpoint added to support the email verification flow. Minimum criteria for release: all 4 test cases pass.

4.2.1 TC6.1 — Valid code within TTL activates the account

Input: POST body { email: “user@test.com”, code: <valid 6-digit code> } submitted within the TTL window. Expected output: 200 OK; account marked as verified in the database. Pass criteria: status 200 and user.verified === true in the database.

4.2.2 TC6.2 — Wrong code returns 400

Input: POST body { email: “user@test.com”, code: “000000” } where the correct code is different. Expected output: 400 Bad Request. Pass criteria: status 400.

4.2.3 TC6.3 — Expired code returns 400

Input: POST body with a valid code whose TTL has elapsed (short TTL used in test environment). Expected output: 400 Bad Request with a code-expired message. Pass criteria: status 400.

4.2.4 TC6.4 — Already-verified account returns 409

Input: POST body for a user whose account is already verified. Expected output: 409 Conflict or a 200 no-op with no state change. Pass criteria: status 409 or 200 with no change to the database.

4.3 User Login (POST /api/auth/login)

Login authenticates a returning user by verifying their credentials, checking that their account is email-verified, and issuing a JWT stored in an HTTP-only cookie. Minimum criteria for release: all 6 test cases pass.

4.3.1 TC7.1 — Valid credentials for a verified account returns 200 and sets JWT cookie

Input: POST body { email: “user@test.com”, password: “securePass1” } for a verified user. Expected output: 200 OK; user object returned; Set-Cookie header present with JWT. Pass criteria: status 200 and the response sets an HTTP-only cookie.

4.3.2 TC7.2 — Missing email returns 400

Input: POST body { password: “securePass1” } with no email. Expected output: 400 Bad Request. Pass criteria: status 400.

4.3.3 TC7.3 — Missing password returns 400

Input: POST body { email: “user@test.com” } with no password. Expected output: 400 Bad Request. Pass criteria: status 400.

4.3.4 TC7.4 — Wrong password returns 400

Input: POST body { email: “user@test.com”, password: “wrongPass99” }. Expected output: 400 Bad Request. Pass criteria: status 400.

4.3.5 TC7.5 — Email not registered returns 404

Input: POST body { email: “ghost@test.com”, password: “securePass1” }. Expected output: 404 Not Found. Pass criteria: status 404.

4.3.6 TC7.6 — Valid credentials but unverified email returns 403

Input: valid credentials for a user who has not completed email verification. Expected output: 403 Forbidden. Pass criteria: status 403 and no JWT cookie is set.

4.4 User Logout (POST /api/auth/logout)

Logout clears the JWT cookie and ends the authenticated session. Minimum criteria for release: the 1 test case passes.

4.4.1 TC8.1 — Logged-in user logs out and JWT cookie is cleared

Input: POST /api/auth/logout with a valid JWT cookie. Expected output: 200 OK; the Set-Cookie header clears the JWT cookie (maxAge set to 0 or the cookie is expired). Pass criteria: status 200 and the JWT cookie is cleared in the response.

4.5 Get User Info (GET /api/auth/userinfo)

Returns the authenticated user’s profile data. This is a protected route requiring a valid JWT cookie. Minimum criteria for release: all 3 test cases pass.

4.5.1 TC9.1 — Valid JWT cookie returns correct user object

Input: GET request with a valid JWT cookie. Expected output: 200 OK; user object containing id, email, firstName, lastName, profileSetup, and color. Pass criteria: status 200 and all fields match the seeded user.

4.5.2 TC9.2 — No cookie returns 401 or 404

Input: GET request with no cookie. Expected output: 401 Unauthorized or 404 Not Found. Pass criteria: status 401 or 404.

4.5.3 TC9.3 — Tampered cookie returns 401 or 403

Input: GET request with a manually altered JWT cookie. Expected output: 401 Unauthorized or 403 Forbidden. Pass criteria: status 401 or 403.

4.6 Update Profile (POST /api/auth/update-profile)

Allows the authenticated user to set their first and last name and an optional color. Once firstName and lastName are set, profileSetup must be set to true. Minimum criteria for release: all 5 test cases pass.

4.6.1 TC10.1 — Valid firstName and lastName updates profile and sets profileSetup: true

Input: POST body { firstName: “Gagandeep”, lastName: “Bhatia” } with valid cookie. Expected output: 200 OK; updated user object with profileSetup: true. Pass criteria: status 200 and body.profileSetup === true.

4.6.2 TC10.2 — No color provided stores the default neutral color

Input: POST body { firstName: “Gagandeep”, lastName: “Bhatia” } with no color field. Expected output: 200 OK; body.color is a non-empty default value. Pass criteria: body.color is not null, undefined, or empty string.

4.6.3 TC10.3 — Providing a color stores it correctly

Input: POST body { firstName: “Gagandeep”, lastName: “Bhatia”, color: “#ff5733” }. Expected output: 200 OK; body.color === “#ff5733”. Pass criteria: the color field in the response matches the submitted value.

4.6.4 TC10.4 — Missing firstName returns 400

Input: POST body { lastName: “Bhatia” } with no firstName. Expected output: 400 Bad Request. Pass criteria: status 400.

4.6.5 TC10.5 — Missing lastName returns 400

Input: POST body { firstName: “Gagandeep” } with no lastName. Expected output: 400 Bad Request. Pass criteria: status 400.

4.7 Search Contacts (POST /api/contacts/search)

Searches all users by name or email and returns matching results, excluding the currently authenticated user. Minimum criteria for release: all 4 test cases pass.

4.7.1 TC11.1 — Valid searchTerm matching a user returns that user

Input: POST body { searchTerm: “John” } with John Doe seeded in the database. Expected output: 200 OK; contacts array containing John Doe. Pass criteria: status 200 and contacts.length >= 1 containing the matching user.

4.7.2 TC11.2 — Valid searchTerm matching no users returns empty array

Input: POST body { searchTerm: “zzznomatch” }. Expected output: 200 OK; contacts: []. Pass criteria: status 200 and contacts.length === 0.

4.7.3 TC11.3 — Missing searchTerm returns 400

Input: POST body {}. Expected output: 400 Bad Request. Pass criteria: status 400.

4.7.4 TC11.4 — Authenticated user is excluded from their own search results

Input: POST body { searchTerm: “Gagandeep” } sent as Gagandeep. Expected output: 200 OK; contacts array does not contain the requesting user. Pass criteria: contacts.every(c => c._id !== requestingUser._id).

4.8 Get All Contacts (GET /api/contacts/all-contacts)

Returns all registered users except the authenticated user, formatted with label (full name) and value (userId). Minimum criteria for release: both test cases pass.

4.8.1 TC12.1 — Returns all users except the requesting user with correct shape

Input: GET request with valid cookie; 3 other users seeded in database. Expected output: 200 OK; contacts array with 3 entries each having label and value fields. Pass criteria: status 200, contacts.length === 3, and each entry has label and value.

4.8.2 TC12.2 — No JWT cookie returns 401

Input: GET request with no cookie. Expected output: 401 Unauthorized. Pass criteria: status 401.

4.9 Get Contacts for List (GET /api/contacts/get-contacts-for-list)

Returns users with whom the authenticated user has an active (non-deleted) DM, sorted by lastMessageTime descending. Minimum criteria for release: all 3 test cases pass.

4.9.1 TC13.1 — Returns only active DM contacts sorted by lastMessageTime descending

Input: GET with valid cookie; User B messaged earlier, User C messaged more recently. Expected output: 200 OK; contacts: [User C, User B]. Pass criteria: status 200 and contacts[0] is User C (most recent).

4.9.2 TC13.2 — Deleted DM contact does not appear in results

Input: GET with valid cookie after the requesting user previously deleted DM with User D. Expected output: 200 OK; User D is absent from the contacts array. Pass criteria: contacts.every(c => c._id !== UserD._id).

4.9.3 TC13.3 — No JWT cookie returns 401

Input: GET request with no cookie. Expected output: 401 Unauthorized. Pass criteria: status 401.

4.10 Delete Direct Messages (DELETE /api/contacts/delete-dm/:dmId)

Removes a DM conversation from the requesting user's view only. Messages remain in the database permanently. The other party retains full visibility of the conversation. Minimum criteria for release: all 5 test cases pass.

4.10.1 TC14.1 — Valid dmId removes conversation from User A's contact list only

Input: DELETE /api/contacts/delete-dm/:userBId with User A's valid cookie. Expected output: 200 OK; User B no longer appears in User A's get-contacts-for-list results. Pass criteria: status 200 and the subsequent contacts-for-list call for User A excludes User B.

4.10.2 TC14.2 — Messages still exist in the database after deletion

Input: DELETE DM between User A and User B; then query the messages collection directly. Expected output: message documents still present in MongoDB. Pass criteria: a direct DB query for messages involving A or B returns more than zero documents.

4.10.3 TC14.3 — User B can still retrieve the conversation after User A deletes it

Input: User A deletes DM; User B calls POST /api/messages/get-messages with User A's ID. Expected output: 200 OK; User B receives the full message history. Pass criteria: status 200 and messages.length > 0 for User B.

4.10.4 TC14.4 — Invalid or missing dmId returns 400

Input: DELETE /api/contacts/delete-dm/ with no dmId parameter or a malformed id. Expected output: 400 Bad Request. Pass criteria: status 400.

4.10.5 TC14.5 — No JWT cookie returns 401

Input: DELETE request with no cookie. Expected output: 401 Unauthorized. Pass criteria: status 401.

4.11 Get Messages (POST /api/messages/get-messages)

Retrieves the full message history between the authenticated user and another user identified by contactId. Returns an empty array if no messages exist. Minimum criteria for release: all 4 test cases pass.

4.11.1 TC15.1 — Valid contactId with existing messages returns correct array

Input: POST body { id: “userBId” } with User A’s cookie; messages seeded in DB. Expected output: 200 OK; messages array with correct sender, recipient, content, and timestamp fields. Pass criteria: status 200, messages.length > 0, and each message has all required fields.

4.11.2 TC15.2 — Valid contactId with no messages returns empty array

Input: POST body { id: “userCId” } where no messages exist between User A and User C. Expected output: 200 OK; messages: []. Pass criteria: status 200 and messages.length === 0.

4.11.3 TC15.3 — Missing contactId returns 400

Input: POST body {}. Expected output: 400 Bad Request. Pass criteria: status 400.

4.11.4 TC15.4 — No JWT cookie returns 401

Input: POST request with no cookie. Expected output: 401 Unauthorized. Pass criteria: status 401.

4.12 Delete Message (DELETE /api/messages/:messageId)

Permanently removes a specific message from MongoDB. Only the sender may delete it. After deletion, neither the sender nor the recipient can retrieve the message via get-messages. This is a new endpoint added beyond the original API spec. Minimum criteria for release: all 5 test cases pass.

4.12.1 TC16.1 — Sender deletes their own message and it is removed from the database

Input: DELETE /api/messages/:messageId with the sender’s cookie. Expected output: 200 OK; message document no longer exists in MongoDB. Pass criteria: status 200 and a direct DB query for the messageId returns null.

4.12.2 TC16.2 — Neither sender nor recipient can retrieve the deleted message

Input: after deletion, both User A and User B call get-messages. Expected output: 200 OK for both; deleted message absent from both arrays. Pass criteria: neither messages array contains the deleted messageId.

4.12.3 TC16.3 — Non-sender attempting to delete returns 403

Input: DELETE /api/messages/:messageId using the recipient's cookie (not the sender). Expected output: 403 Forbidden. Pass criteria: status 403 and the message still exists in the database.

4.12.4 TC16.4 — Non-existent messageId returns 404

Input: DELETE /api/messages/nonexistentId with a valid cookie. Expected output: 404 Not Found. Pass criteria: status 404.

4.12.5 TC16.5 — No JWT cookie returns 401

Input: DELETE request with no cookie. Expected output: 401 Unauthorized. Pass criteria: status 401.

5 System Testing

System testing validates the application as a whole by testing complete multi-step user flows end-to-end, verifying real-time WebSocket behavior, and probing the system for security vulnerabilities. Unlike feature tests which test each endpoint in isolation, system tests exercise the full stack of middleware, routing, database, and socket layers together. All system tests use Jest with Supertest and live Socket.IO client connections against the full Express and Socket.IO server backed by an in-memory MongoDB instance.

5.1 End-to-End User Flow Tests

These tests simulate complete multi-step user journeys to verify that all components work together correctly in sequence. Minimum criteria for release: all 3 test cases pass.

5.1.1 TC17.1 — Full signup → verify → login → update profile flow completes successfully

Input: step 1 POST /signup, step 2 POST /verify-email, step 3 POST /login, step 4 POST /update-profile. Expected output: each step returns the correct status code and the user ends with profileSetup: true and a valid session. Pass criteria: all 4 steps return their expected statuses and the final userinfo call shows the updated profile.

5.1.2 TC17.2 — Full messaging flow: login → send via socket → retrieve via REST

Input: two users log in, User A emits a sendMessage socket event targeting User B, then User B calls get-messages. Expected output: the message is saved to the database and User B's get-messages response contains it. Pass criteria: get-messages returns the message sent via socket.

5.1.3 TC17.3 — Delete DM flow: User A deletes, User B retains full history

Input: A and B exchange messages, A calls delete-dm, then both call get-contacts-for-list and get-messages. Expected output: A's contact list excludes B; B's contact list still

includes A; B can retrieve the full message history; A can no longer see the conversation. Pass criteria: correct visibility for both users post-deletion.

5.2 WebSocket / Socket.IO Tests

These tests validate real-time messaging behavior using live Socket.IO client connections. Two authenticated socket clients are connected in beforeAll and socket events are asserted using event listeners. Minimum criteria for release: all 6 test cases pass.

5.2.1 TC18.1 — Authenticated client connects and connection is accepted

Input: Socket.IO client connects with a valid JWT cookie in the handshake. Expected output: connection is established; the connect event fires with no error. Pass criteria: socket.connected === true.

5.2.2 TC18.2 — Client with no JWT cookie is rejected

Input: Socket.IO client connects with no cookie in the handshake. Expected output: connection is refused; an error event fires or socket.connected === false. Pass criteria: socket.connected === false or an error is received.

5.2.3 TC18.3 — Client with tampered JWT cookie is rejected

Input: Socket.IO client connects with an altered JWT cookie. Expected output: connection is refused. Pass criteria: socket.connected === false or an error is received.

5.2.4 TC18.4 — sendMessage saves to DB and emits receiveMessage to both parties

Input: User A emits sendMessage { sender: A, recipient: B, content: “Hello” }. Expected output: a message document is created in MongoDB and both User A and User B receive a receiveMessage event with the correct payload. Pass criteria: the database contains the message and both sockets receive the event.

5.2.5 TC18.5 — receiveMessage payload contains all required fields

Input: sendMessage emitted by User A. Expected output: the receiveMessage payload contains id, sender (object), recipient (object), content, messageType, and timestamp. Pass criteria: all 6 fields are present on the received payload.

5.2.6 TC18.6 — Message is persisted when recipient is offline

Input: User B has no active socket connection; User A emits sendMessage targeting User B. Expected output: the message is saved to the database; User B can retrieve it via POST /api/messages/get-messages after reconnecting. Pass criteria: the database contains the message and get-messages for User B returns it. This test verifies DB persistence only, not real-time socket delivery to offline users.

5.3 Security Tests

Security testing validates that the backend is resilient against common web vulnerabilities as required by the course specification. Tests cover JWT enforcement on all protected routes, NoSQL injection resistance, XSS payload handling, session hijacking considerations, and brute force protection as a stretch goal.

5.3.1 TC19.1 — All protected routes return 401 without a JWT cookie

Input: one request to each protected route with no cookie: /userinfo, /update-profile, /contacts/search, /contacts/all-contacts, /contacts/get-contacts-for-list, /contacts/delete-dm/:id, /messages/get-messages, /messages/:id. Expected output: 401 Unauthorized for every route. Pass criteria: all 8 routes return 401.

5.3.2 TC19.2 — All protected routes return 401 or 403 with an expired JWT

Input: a token generated with an expiry of -1 second used on each protected route. Expected output: 401 or 403 for every route. Pass criteria: all protected routes reject the expired token.

5.3.3 TC19.3 — All protected routes return 401 or 403 with a tampered JWT

Input: a valid JWT with its last character altered, used on each protected route. Expected output: 401 or 403 for every route. Pass criteria: all protected routes reject the tampered token.

5.3.4 TC19.4 — NoSQL injection in login email does not authenticate

Input: POST /login body { email: { “$gt”: “” }, password: “anything” }. Expected output: 400 Bad Request or 404 Not Found; no JWT cookie set. Pass criteria: status 400 or 404 and no cookie is set in the response. The server must not return 200.

5.3.5 TC19.5 — NoSQL injection in login password does not authenticate

Input: POST /login body { email: “user@test.com”, password: { “$gt”: “” } }. Expected output: 400 Bad Request; no JWT cookie set. Pass criteria: status 400 and no cookie is set.

5.3.6 TC19.6 — NoSQL injection in search contacts does not crash the server

Input: POST /contacts/search body { searchTerm: { “$gt”: “” } } with valid cookie. Expected output: 400 Bad Request or 200 with empty array; server does not crash. Pass criteria: status 400 or 200; no 500 Internal Server Error.

5.3.7 TC19.7 — XSS payload in profile fields is stored as plain text

Input: POST /update-profile body { firstName: “<script>alert(1)</script>”, lastName: “Test” }. Expected output: 200 OK; body.firstName is the literal string stored without any modification or execution. Pass criteria: body.firstName === “<script>alert(1)</script>” as a plain string.

5.3.8 TC19.8 — XSS payload in message content is stored as plain text

Input: sendMessage socket event with content “<script>alert(1)</script>”. Expected output: the message is stored in the database and returned via get-messages as a literal string. Pass criteria: message.content === “<script>alert(1)</script>” as a plain string.

5.3.9 TC19.9 — [STRETCH GOAL] Brute force protection returns 429 after N failed logins

Input: more than the configured threshold (e.g. 10) POST /login requests with wrong credentials in rapid succession from the same IP. Expected output: 429 Too Many Requests returned on the (N+1)th attempt. Pass criteria: status 429 is returned once the threshold is exceeded. Note: this is a stretch goal. If rate limiting is not implemented, this test case must be documented as a known security gap with a written rationale.

If all test results have met the release criteria defined in Section 2.3, the system is ready for deployment and integration with the provided frontend.